Module Code	**:**	CT124-3-1-ICS – Integrated Computer Systems
Intake Code	**:**	APU1F2507CS(CYB)
Lecturer Name	**:**	VIJAYALAXMI AMUTHAN
Hand in Date	**:**	April 2026
Tutorial No.		T-3
Group No.	**:**	Group 37
Group Leader	**:**	Istiaque Ahmed Ifty

Student ID	**Student Name**
TP089609	Donaley Kururama Makarawa
TP090011	Istiaque Ahmed Ifty
TP092108	Mohtasim Ahmed Rhythm
TP089754	Royyan Firdaus Alpha
TP091919	W Shein Zi

Table of Contents

1.0 INTRODUCTION

This report presents the design and implementation of menu-driven NASM Assembly 32-bit shape generation program. The program algorithmically generates five unique shapes using mathematical loops, parameters, and logic-based boundary conditions. Each shape is contributed by one member of the shape which is then integrated into a single unified program. This program also satisfies the dynamic execution requirements by having different randomization factors on each shape. Section A covers system architecture, hardware interaction, input validation, error handling, and each shape's drawing algorithm. Section B provides a research analysis comparing this assembly program to an equivalent python program. Furthermore, it examines how random outputs affect overall program performance and efficiency. At last, proposals were given to improve the program by implementing Artificial Intelligence or Machine Learning enhancements.

2.0 SECTION A: ASSEMBLY PROGRAMMING

2.1 System Architecture and Hardware Interaction

This section includes the management of memory and execution of input/output operations inside the Linux kernel and system hardware.

2.1.1 MEMORY ALLOCATION AND DATA INITIALIZATION

This block handles all the prompt messages, newlines, and spaces needed to write input/output and print spaces when needed while drawing the desired shape. It also uses dynamically stored memory buffers to optimize the whole program.

```
section .data
    ; Main Menu
    mainmenu_msg db 10, "NASM ASSEMBLY SHAPE GENERATOR", 10
                 db "1. Concentric Square (Istiaque Ahmed Ifty)", 10
                 db "2. Hexagon (Mohtasim Ahmed Rhythm)", 10
                 db "3. Hourglass (Donaley Kururama Makarawa)", 10
                 db "4. Diamond (W Shein Zi)", 10
                 db "5. Heart (Royyan Firdaus Alpha)", 10
                 db "6. Exit", 10
                 db "Select Option(1-6): ", 0
    mainmenu_msg_len equ $ - mainmenu_msg

    ; All of the prompts for the program
    prompt_size db "Enter size (1-9): ", 0
    prompt_size_len equ $ - prompt_size     ; Calculates the length of the prompt
    prompt_location db "Enter location (1=Left, 2=Center, 3=Right): ", 0
    prompt_location_len equ $ - prompt_location
    prompt_number db "Enter number of shapes (1-9): ", 0
    prompt_number_len equ $ - prompt_number
    prompt_character db "Enter drawing character (e.g. #, %, *): ", 0
    prompt_character_len equ $ - prompt_character
    prompt_color db "Select Color (1=Red, 2=Green, 3=Yellow, 4=Blue, 5=Magenta, 6=Cyan, 7=Reset): ", 0
    prompt_color_len equ $ - prompt_color
```

```
    ; Error message for input validation
    error_msg db "INVALID INPUT! Please select a valid option."
    error_msg_len equ $ - error_msg

    ; Ansi color codes
    c_red         db  27, "[31m", 0
    c_green       db  27, "[32m", 0
    c_yellow      db  27, "[33m", 0
    c_blue        db  27, "[34m", 0
    c_magenta     db  27, "[35m", 0
    c_cyan        db  27, "[36m", 0
    c_reset       db  27, "[0m",  0

    ; Specific Randomization Announcements
    msg_color db 10, "[>> Applying Random Color...]", 10, 0
    msg_color_len equ $ - msg_color
    msg_size db 10, "[>> Generating Random Size ...]", 10, 0
    msg_size_len equ $ - msg_size
    msg_loc db 10, "[>> Shuffling Random Location...]", 10, 0
    msg_loc_len equ $ - msg_loc
    msg_char db 10, "[>> Picking Random Character...]", 10, 0
    msg_char_len equ $ - msg_char
    msg_num db 10, "[>> Rolling Random Number of Shapes...]", 10, 0
    msg_num_len equ $ - msg_num

    newline db 10, 0
    space db "  ", 0
```

- The **.data** segement reserves memory to show input/output depending on specific functions and requirements inside the program.

- The **db** command allocates exactly 8 bits of memory per character.
- The **10** value works as a newline.
- **0** marks the end of the string.
- The **equ $ -** symbols are used to calculate the length of the string. **"$"** means the current memory address of the string and **"- msg"** is the starting memory address of the message.
- ANSI Color codes such as "**27**" triggers terminal color to change natively, and other codes(e.g. **31m, 34m**) are used to generate different colors according to user input.

```
section .bss
    ; Shared variables to save memory
    input_user resb 16                  ; Reserves 16 bytes to securely absorb buffer overflows
    max_size resd 1                     ; Reserves 4 bytes or 1 dword
    x_offset resd 1                     ; Memory address for location change
    num_of_shapes resd 1                ; Memory address for number of squares/shapes
    draw_character resb 2               ; Reserves 2 bytes for any character to be drawn
    chosen_color resd 1                 ; Reserves 4 bytes or 1 dword for the randomized color
    current_row resd 1                  ; Memory address to use in the row loop
    current_col resd 1                  ; Memory address to use in the col loop
    space_cnt resd 1                    ; Memory address to print spaces

    ; Hexagon specific math variables
    rand_size resd 1                    ; Memory address for the size value that will be randomized
    max_width resd 1                    ; Memory address for the width value that is necessary for hexagon drawing calculation
```

- **.bss** section reserves ram memory to store various inputs to run loops, error handling, and overall program execution.
- The **resb** coomand reserves 1 byte naturally, but here 16 bytes are reserved to securely absorb buffer overflows from the user input.
- **resd** reserves 4 bytes or 1 dword.
- Variables like **max_size** and **x_offset** is used by all 5 shapes algorithm.

2.1.2 PROGRAM ENTRY AND MAIN MENU

```
section .text
    global _start

_start:
```

- The **.text** Segment is where the actual CPU instructions are stored.
- **global _start** is a linker directive to let Linux know where the program starts.

main_menu:

```
; Main menu function
main_menu:
    mov ecx, mainmenu_msg               ; collects the memory address for the main menu message
    mov edx, mainmenu_msg_len           ; gets the length of the memory address
    call print_msg                      ; calls the sys_write subroutine function
    call read_input                     ; calls the sys_read subroutine function

    ; Main menu error handling
    cmp eax, 2                          ; Hardware Check: 1 input char + 1 Enter key = Exactly 2 bytes
    jne error_main                      ; Rejects anything longer, or empty Enter keys

    xor eax, eax
    mov al, [input_user]
    cmp al, '1'
    je init_square
    cmp al, '2'
    je init_hexagon
    cmp al, '3'
    je init_hourglass
    cmp al, '4'
    je init_diamond
    cmp al, '5'
    je init_heart
    cmp al, '6'
    je exit
    jmp error_main
```

- The CPU register known as **mov ecx** locates the memory address for the main menu message prompt.
- The CPU register known as **mov edx** gathers the length of the memory address.
- The **call print_msg** and **read_input**, jumps to shared input/output system call subroutines to print prompts and read inputs from the user.
- Main menu error handling is done by **cmp eax, 2** which validates that the user pressed exactly one character and “Enter” key (2 bytes in total).
- If the input length is anything other than 2 bytes, it rejects the input and jumps to the input validation function known as **error_main**.
- **xor eax, eax** ensures that **eax** register is wiped before taking inputs, otherwise **eax** register can get easily overwritten as wrong input.
- **mov al** takes exactly 1 byte into **eax** register from the user input.
- **cmp al** compares keystrokes pressed by the user and jumps according to the specific initialization function.
- **je** checks if the comparison matches, then it jumps to the given functions.
- **jmp error_main** checks any of the condition matches the 1-6 parameters, if not then it hits the error function immediately.

2.1.3 SHAPE INITIALIZATION

Before looping through each drawing logic, the program sets an identifier to detect which drawing algorithm to run.

```
; Different register for each shapes logic and loop
init_square:
    mov bl, 1                               ; bl = 1 means Square flow
    jmp ask_size

init_hexagon:
    mov bl, 2                               ; bl = 2 means Hexagon flow
    jmp ask_location

init_hourglass:
    mov bl, 3                               ; bl = 3 means Hourglass flow
    jmp ask_size

init_diamond:
    mov bl, 4                               ; bl = 4 means Diamond flow
    jmp ask_size

init_heart:
    mov bl, 5                               ; bl = 5 means Heart flow
    jmp ask_size

; SHARED INPUT COLLECTION FUNCTIONS
```

- The **bl** Register is a global identifier to track which shape drawing algorithm is active.
- **jmp**, jumps to certain input gathering functions according to user input.

2.1.4 SHARED INPUT COLLECTION FUNCTIONS

Every shape drawing algorithm uses universal shared input collection function to optimize overall code.

ask_size: , ask_number:

```
; SHARED INPUT COLLECTION FUNCTIONS
ask_size:
    mov ecx, prompt_size
    mov edx, prompt_size_len
    call print_msg                          ; calls the print_msg subroutine function to print
    call read_input                         ; calls the read_input subroutine function to read input

    ; error handles double digit number
    cmp eax, 2
    jne error_size

    ; Error handling for the size input
    xor eax, eax                            ; wipes the eax register for error handling
    mov al, [input_user]
    cmp al, '1'
    jl error_size
    cmp al, '9'
    jg error_size

    ; Storing the input size into memory
    sub al, '0'                             ;subtracts 48 which is 0 according to ASCII to convert string to interger
    mov [max_size], eax

    cmp bl, 3                               ; checks if hourglass skips for location input
    je ask_number                           ; it skips because hourglass randomizes location instead
    jmp ask_location
```

```
ask_number:
    mov ecx, prompt_number
    mov edx, prompt_number_len
    call print_msg
    call read_input

    ; Input Validation & Sanitization
    cmp eax, 2
    jne error_number

    ; Error handling for asking number of shapes
    xor eax, eax
    mov al, [input_user]
    cmp al, '1'
    jl error_number
    cmp al, '9'
    jg error_number

    ; Data collection for how many number of shapes
    sub al, '0'
    mov [num_of_shapes], eax

    cmp bl, 4
    je ask_color                        ; diamond skips asking character (randomizes it)
```

- **mov ecx, edx** are used to gather location and length of the given prompt. Additionally, **call** is used to jump to **sys_write** and **sys_read** subroutines.
- It utilizes the same 2-byte input validation as main menu function.
- **xor eax, eax** wipes the register clean and puts the input into **mov al** to error handle. It compares as if it is less than 1 or greater than 9, then it jumps to the error functions.
- **sub al, 0** subtract 48 from the register which converts the string into raw math data.
- **mov[max_size], eax** and **mov[num_of_shapes], eax** moves the data from **eax** register to the **.bss** ram for further use in the future.
- **cmp bl, 3** checks it hourglass shape is active. If it is active, it skips asking for location because hourglass generates random location output. Additionally, **cmp bl, 4** checks if diamond shape is currently active or not. If it is active, then it skips **ask_character** and jumps to **ask_color**.

ask_location:

```
ask_location:
    mov ecx, prompt_location
    mov edx, prompt_location_len
    call print_msg
    call read_input

    ; Input Validation & Sanitization
    cmp eax, 2
    jne error_location

    ; Error handling for location input
    xor eax, eax
    mov al, [input_user]
    cmp al, '1'
    je set_left
    cmp al, '2'
    je set_center
    cmp al, '3'
    je set_right
    jmp error_location

; Data storing for location changes
```

- **mov ecx, edx** locates the location and length of the **prompt_location** memory address.
- The **call** instructions lead to the universal **sys_write** and **sys_read** system calls.
- It uses same error handling as any more than 2 bytes of data inserted is rejected.
- **Cmp** and **je** checks whether the input is from 1 to 3, otherwise it jumps to **error_location** function.

```
; Data storing for location changes
set_left:
    mov dword [x_offset], 0
    cmp bl, 5                           ; checks if heart skips number of shapes input
    je ask_character                    ; it skips because heart randomizes number of shapes
    jmp ask_number

set_center:
    mov dword [x_offset], 20
    cmp bl, 5
    je ask_character
    jmp ask_number

set_right:
    mov dword [x_offset], 40
    cmp bl, 5
    je ask_character
    jmp ask_number
```

- **s**Moves 0 to the **x_offset** memory address to print the certain shape left of the terminal. **cmp bl, 5** checks if it is the heart shape, then it skips if it is, otherwise jumps to the next function.
- **set_center Allocation:** Moves 20 to the **x_offset** memory address to print the specific shape center of the terminal.
- Moves 40 to the **x_offset** memory address to print the specific shape right of the terminal.

ask_character:

```
ask_character:
    mov ecx, prompt_character
    mov edx, prompt_character_len
    call print_msg
    call read_input

    cmp eax, 2
    jne error_character

    mov al, [input_user]
    mov [draw_character], al
    mov byte [draw_character + 1], ' '  ; adds a space after the symbol to make the drawn pixels twice as wide

    cmp bl, 1
    je sq_announce                      ; if square, then announce color and jump to randomizer
    jmp ask_color                       ; if hexagon/hourglass/heart/diamond, then ask for color
```

- Re-uses previously mentioned **sys_write** and **sys_read** subroutines and 2-byte error handling **(cmp eax, 2)** to maintain program integrity.
- Uses **al** register to extract only 1 byte for calculation.
- **mov[draw_character])** saves the input character into **.bss** ram for further use.
- The **mov byte** command inserts a blank space next to the character, making the printed symbol twice as wide, ensuring that the shapes look like a perfect square.
- **cmp bl, 1** check whether the square shape is active. If active, it then jumps to drawing algorithm rather asking for color.

ask_color:

```
ask_color:
    mov ecx, prompt_color
    mov edx, prompt_color_len
    call print_msg
    call read_input

    cmp eax, 2
    jne error_color

    mov al, [input_user]
    cmp al, '1'
    je set_red
    cmp al, '2'
    je set_green
s    cmp al, '3'
    je set_yellow
    cmp al, '4'
    je set_blue
    cmp al, '5'
    je set_magenta
    cmp al, '6'
    je set_cyan
    cmp al, '7'
    je set_default
    jmp error_color
```

```
set_red:
    mov dword [chosen_color], c_red
    cmp bl, 1
    je sq_master_loop
    cmp bl, 2
    je hex_announce
    cmp bl, 3
    je hg_announce
    cmp bl, 4
    je dia_announce
    jmp hrt_announce

set_green:
    mov dword [chosen_color], c_green
    cmp bl, 1
    je sq_master_loop
    cmp bl, 2
    je hex_announce
    cmp bl, 3
    je hg_announce
    cmp bl, 4
    je dia_announce
    jmp hrt_announce

set_blue:
    mov dword [chosen_color], c_blue
    cmp bl, 1
    je sq_master_loop
    cmp bl, 2
    je hex_announce
    cmp bl, 3
    je hg_announce
    cmp bl, 4
    je dia_announce
    jmp hrt_announce
```

```
set_yellow:
    mov dword [chosen_color], c_yellow
    cmp bl, 1
    je sq_master_loop
    cmp bl, 2
    je hex_announce
    cmp bl, 3
    je hg_announce
    cmp bl, 4
    je dia_announce
    jmp hrt_announce

set_cyan:
    mov dword [chosen_color], c_cyan
    cmp bl, 1
    je sq_master_loop
    cmp bl, 2
    je hex_announce
    cmp bl, 3
    je hg_announce
    cmp bl, 4
    je dia_announce
    jmp hrt_announce

set_magenta:
    mov dword [chosen_color], c_magenta
    cmp bl, 1
    je sq_master_loop
    cmp bl, 2
    je hex_announce
    cmp bl, 3
    je hg_announce
    cmp bl, 4
    je dia_announce
    jmp hrt_announce

set_default:
    mov dword [chosen_color], c_reset
    cmp bl, 1
    je sq_master_loop
    cmp bl, 2
    je hex_announce
    cmp bl, 3
    je hg_announce
    cmp bl, 4
    je dia_announce
    jmp hrt_announce
```

Code Analysis:

- Re-applies the calling of **sys_write** and **sys_read** subroutines functions and the 2-byte hardware check **(cmp eax, 2)** to error handle.
- **cmp al** checks user input from 1 to 7 and routes the CPU to the corresponding color functions (e.g. **set_red, set_green**).
- **mov dword** loads the memory address of the selected ANSI escape code string (e.g. **c_red**) into the shared **chosen_color** variabl.
- **cmp bl** makes sure that instead of printing color menu five times, it checks bl register which acts like a tag to remember which shape user chose at the main menu. After saving the color, it reads bl menu and jumps back to the exact shape that asked for it.

2.1.5 SHARED ERROR HANDLING FUNCTIONS

The whole program uses one shared error handling function to optimize the code structure and execution.

```
; SHARED INPUT VALIDATION FUNCTIONS
error_main:
    call print_error_msg
    jmp main_menu

error_size:
    call print_error_msg
    jmp ask_size

error_location:
    call print_error_msg
    jmp ask_location

error_number:
    call print_error_msg
    jmp ask_number

error_character:
    call print_error_msg
    jmp ask_character

error_color:
    call print_error_msg
    jmp ask_color

print_error_msg:
    mov ecx, error_msg
    mov edx, error_msg_len
    call print_msg

    mov ecx, newline
    mov edx, 1
    call print_msg

    ret                             ; teleports backs to whichever error block called it
```

- Here different dedicated functions (e.g. **error_main, error_size**) are used to error handle specific inputs from the user. After validating input, it jumps to the specific function where the error occurred.
- Instead of using different error messages, this program uses one universal error message by calling **print_error_msg** function.
- The **ret** command at the end of the function dynamically teleports back to whichever error block originally called it.

2.1.6 SHARED SUBROUTINE & EXIT FUNCTIONS

This program uses universal **sys_write** and **sys_read** subroutine function to reduce overall code redundance.

```
; SHARED SUBROUTINE FUNCTIONS
print_msg:
    push eax                        ; pushes into stack to protect eax
    push ebx                        ; pushes into stack to protect ebx
    mov eax, 4                      ; sys_write
    mov ebx, 1                      ; stdout
    int 0x80                        ; System call for linux 32 bit
    pop ebx                         ; restores ebx
    pop eax                         ; restores eax
    ret                             ; returns to the line of code where print_msg was called

read_input:
    ; Linux overwrites eax with the keystroke count which we need for input validation.
    push ebx                        ; pushes into stack to protect ebx
    push ecx                        ; pushes into stack to protect ecx
    push edx                        ; pushes into stack to protect edx
    mov eax, 3                      ; sys_read for terminal
    mov ebx, 0                      ; stdin
    mov ecx, input_user             ; collects the input of the user
    mov edx, 16                     ; clears up to 16 bytes from the OS buffer
    int 0x80
    pop edx                         ; restores edx
    pop ecx                         ; restores ecx
    pop ebx                         ; restores ebx
    ret                             ; returns to the line of code where read_input was called

; EXIT FUNCTION
exit:
    mov eax, 1
    xor ebx, ebx
    int 0x80
```

- **mov eax, 4** is used for printing text and **mov eax, 3** is used for collecting user input. Also, **int0x80** is used to trigger these commands.
- The **push** & **pop** command are used to protect CPU register from overwriting them. The push commands save shape-drawing coordinates while pop puts them right back afterwards, so the program doesn't break..
- The read function is designed to absorb up to 16 bytes of inputs at a time **(mov edx, 16)**. This prevents the program from crashing randomly if the user starts mashing keyboard.
- **mov eax, 1** is used for **sys_exit** to exit the program while **xor ebx, ebx** sets the status code to 0, which tells the operating system the program ran successfully with no errors.

2.2 Shape Generation Algorithms

2.2.1 RANDOMIZATION FUNCTIONS

Different randomization functions are used to randomize color, size, location, character, and number of shapes depending on the shape chosen by the user.

generate_random_color/hex_size/hg_location/dia_char/hrt_num:

```
; ALL RANDOMIZATION FUNCTIONS
generate_random_color:
    push eax                            ; protects eax register before randomization
    push ecx                            ; protects ecx register before randomization
    push edx                            ; protects edx register before randomization

    ; grabs the cpu clock cycles
    rdtsc                               ; fills eax with low bits and edx with high bits
    xor edx, edx                        ; wipes edx register
    mov ecx, 7                          ; sets divisor to 7
    div ecx                             ; divide eax by 7 while the remainder (0-6) drops back into edx.

    ; Routes to the randomized color
    cmp edx, 0
    je rand_set_red                      ; jump to red
    cmp edx, 1
    je rand_set_green
    cmp edx, 2
    je rand_set_blue
    cmp edx, 3
    je rand_set_yellow
    cmp edx, 4
    je rand_set_cyan
    cmp edx, 5
    je rand_set_magenta
    jmp rand_set_default                ; if the remainder is 6 it picks default
```

```
rand_set_red:
    mov dword [chosen_color], c_red     ; applies red color
    jmp color_done                      ; jumps to color_done
rand_set_green:
    mov dword [chosen_color], c_green   ; applies green color
    jmp color_done
rand_set_blue:
    mov dword [chosen_color], c_blue    ; applies blue color
    jmp color_done
rand_set_yellow:
    mov dword [chosen_color], c_yellow  ; applies yellow color
    jmp color_done
rand_set_cyan:
    mov dword [chosen_color], c_cyan    ; applies cyan color
    jmp color_done
rand_set_magenta:
    mov dword [chosen_color], c_magenta ; applies magenta color
    jmp color_done
rand_set_default:
    mov dword [chosen_color], c_reset   ; applies default color
    jmp color_done

color_done:
    pop edx                             ; restores edx register
    pop ecx                             ; restores ecx register
    pop eax                             ; restores eax register
    ret                                 ; returns to master loop
```

```
generate_random_hex_size:
    push eax                            ; protects eax register
    push ecx                            ; protects ecx register
    push edx                            ; protects edx register

    ; randomization of the size value (it juggles from 3 to 9)
    rdtsc                               ; reads time-stamp Counter (CPU clock cycles) which fills eax and edx
    xor edx, edx                        ; wipes the edx register clean for division
    mov ecx, 7                          ; sets the divisor to 7
    div ecx                             ; divides by 7 and the remainder from 0 to 6 goes to edx
    add edx, 3                          ; adds 3 to the remainder. now edx is a number from 3 to 9
    mov [rand_size], edx                ; saves this random number as the main size

    pop edx                             ; restores edx register
    pop ecx                             ; restores ecx register
    pop eax                             ; restores eax register
    ret                                 ; returns to master loop
```

```
generate_random_hg_location:
    push eax                             ; protects eax register
    push ecx                             ; protects ecx register
    push edx                             ; protects edx register

    ; Random Location Generator (Juggles location for every shape drawn)
    rdtsc                                ; grabs CPU clock cycles
    xor edx, edx
    mov ecx, 3                           ; divides by 3
    div ecx                              ; remainder (0, 1, or 2) goes to edx

    ; allocates location according to the randomization factor
    cmp edx, 0
    je rand_left
    cmp edx, 1
    je rand_center
    jmp rand_right

rand_left:
    mov dword [x_offset], 0              ; applies left offset
    jmp hg_loc_done                      ; jumps to loc_done
rand_center:
    mov dword [x_offset], 20             ; applies center offset
    jmp hg_loc_done
rand_right:
    mov dword [x_offset], 40             ; applies right offset
    jmp hg_loc_done

hg_loc_done:
    pop edx                              ; restores edx register
    pop ecx                              ; restores ecx register
    pop eax                              ; restores eax register
    ret                                  ; returns to master loop
```

```
generate_random_dia_char:
    push eax                              ; protects eax register
    push ecx                              ; protects ecx register
    push edx                              ; protects edx register

    ; Random Symbol Generator (Juggles A-Z for every shape drawn)
    rdtsc                                 ; grabs CPU clock cycles
    xor edx, edx
    mov ecx, 94                           ; divide by 94 total character
    div ecx                               ; remainders is  0 to 93
    add edx, 33                           ; add 33 into the printable ascii block
    mov [draw_character], dl              ; loads the random letter into the draw variable
    mov byte [draw_character + 1], ' '

    pop edx                               ; restores edx register
    pop ecx                               ; restores ecx register
    pop eax                               ; restores eax register
    ret                                   ; returns to master loop
```

```
generate_random_hrt_num:
    push eax                            ; protects eax register
    push ecx                            ; protects ecx register
    push edx                            ; protects edx register

    ; Random Number Generator (1 to 9 shapes)
    rdtsc
    xor edx, edx
    mov ecx, 9
    div ecx
    add edx, 1                          ; shift remainder from 0-8 to 1-9
    mov [num_of_shapes], edx

    pop edx                             ; restores edx register
    pop ecx                             ; restores ecx register
    pop eax                             ; restores eax register
    ret                                 ; returns to master loop
```

- **push** protects **eax, ecx,** and **edx** register so they do not overwrite the data inside the main loop.
- **rdtsc** captures exact number of CPU clock cycles since the computer was booted up.
- **xor edx, edx** is used to wipe the edx register clean.
- Since there is 7 colors to choose from, **mov ecx, 7** is used to put number 7 into the **ecx** register to act as the divisor.
- **div ecx** divides the massive CPU clock numbers . In assembly, the **div** commands splits the answer by putting full groups into **eax** and remainders into **edx**. The program ignores the group and uses this remainders in **edx**.
- **cmp edx** is used to randomly select what would be printed which then jumps to desired color printing function(e.g. **rand_set_red, rand_set_green, rand_left)**.
- The **pop** command is used to return the saved register value to **eax, ecx,** and **eax** registers. The **ret** command is used to return to the master loop after applying randomization.

2.2.2 PRE-EXECUTION ANNOUNCEMENTS

Before printing the shape, the terminal announces the randomization factor according to the shape chosen.

```
sq_announce:                            ; announces the randomization factor in the terminal
    mov ecx, msg_color
    mov edx, msg_color_len
    call print_msg
```

```
hex_announce:                           ; announces the randomization factor
    mov ecx, msg_size
    mov edx, msg_size_len
    call print_msg
```

```
hg_announce:                            ; announces the randomization factor
    mov ecx, msg_loc
    mov edx, msg_loc_len
    call print_msg
```

```
dia_announce:                            ; announces the randomization factor
    mov ecx, msg_char
    mov edx, msg_char_len
    call print_msg
```

```
hrt_announce:
    mov ecx, msg_num
    mov edx, msg_num_len
    call print_msg
    call generate_random_hrt_num        ; calls the dedicated subroutine for number of shapes
```

- **mov ecx** and **mov edx** gathers location and length of the memory address needed for each announcement messages.
- **call** jumps to **sys_write** subroutine function to write in the linux terminal.

2.2.3 STANDARDIZED ITERATION FUNCTIONS

To maximize code reusability, every shape algorithm uses an identical set of looping and iteration mechanics.

sq/hex/hg/dia_init_row_loop:

```
sq_init_row_loop:
    mov eax, [max_size]
    neg eax                             ; turns the input negative to make the (0,0) coordinates center
    mov [current_row], eax              ; puts the input size into the current row counter
```

```
hex_init_row_loop:
    mov eax, [rand_size]
    neg eax
    mov [current_row], eax
```

```
; initializes the setup before row loop
hg_init_row_loop:
    mov eax, [max_size]
    neg eax                            ; makes the max_size negative to make (0,0) the center of the hourglass
    mov [current_row], eax             ; Sets the vertical grid
```

```
; initializes the setup before row loop
dia_init_row_loop:
    mov eax, [max_size]
    neg eax
    mov [current_row], eax                ; sets the vertical grid
```

- Initializes the row loop by putting the size inputted by the user into **eax** register through **mov eax, [max_size]**.
- **neg eax** makes the y coordinate negative.

- Then it moves back to **current_row** variable to calculate rows.

sq/hex/hg/dia/hrt_row_loop:

```
sq_row_loop:
    mov eax, [current_row]
    cmp eax, [max_size]                ; compares according to the input Size
    jg sq_shape_done

    ; Location loop initialization
    mov eax, [x_offset]
    mov [space_cnt], eax
```

```
hex_row_loop:
    mov eax, [current_row]
    cmp eax, [rand_size]               ; compares to the randomized size
    jg hex_shape_done                  ; jumps if greater than the randomized size

    ; Location loop intialization
    mov eax, [x_offset]                ; pulls the offset (15, 35, or 55)
    mov [space_cnt], eax
```

```
hg_row_loop:
    mov eax, [current_row]
    cmp eax, [max_size]
    jg hg_shape_done

    ; location loop intialization
    mov eax, [x_offset]
    mov [space_cnt], eax
```

```
dia_row_loop:
    mov eax, [current_row]
    cmp eax, [max_size]
    jg dia_shape_done

    ; location loop intialization
    mov eax, [x_offset]
    mov [space_cnt], eax
```

```
hrt_row_loop:
    mov eax, [current_row]
    cmp eax, [max_size]
    jg hrt_shape_done                  ; if Y > max_size,  jumps to shape done

    ; Push shape to correct location
    mov eax, [x_offset]
    mov [space_cnt], eax
```

- Moves the stored variable in **current_row** into **eax** register to calculate rows.
- Compares it with **max_size** variable using **cmp** command.
- If it is greater than **max_size** then it jumps to **sq_shape_done**.
- Initializes location loop by putting **x_offset** variable into **eax** register and inserting the eax register into **space_cnt** variable to print empty spaces

sq/hex/hg/dia/hrt_location_loop:

```
sq_location_loop:
    cmp dword [space_cnt], 0
    je sq_col_loop_initialization
    mov ecx, space
    mov edx, 1
    call print_msg
    dec dword [space_cnt]
    jmp sq_location_loop
```

```
hex_location_loop:
    cmp dword [space_cnt], 0            ; checks whether we have printed all the offset spaces
    je hex_col_loop_initialization      ; if yes then starts drawing the columns
    mov ecx, space                      ; loads 1 blank space
    mov edx, 1
    call print_msg                      ; prints the space
    dec dword [space_cnt]               ; subtracts 1 from the counter
    jmp hex_location_loop               ; loop again untill it hits 0
```

```
hg_location_loop:
    cmp dword [space_cnt], 0
    je hg_col_loop_initialization
    mov ecx, space
    mov edx, 1
    call print_msg
    dec dword [space_cnt]
    jmp hg_location_loop
```

```
dia_location_loop:
    cmp dword [space_cnt], 0
    je dia_col_loop_initialization
    mov ecx, space
    mov edx, 1
    call print_msg
    dec dword [space_cnt]
    jmp dia_location_loop
```

```
hrt_location_loop:
    cmp dword [space_cnt], 0
    je hrt_col_loop_initialization
    mov ecx, space
    mov edx, 1
    call print_msg
    dec dword [space_cnt]
    jmp hrt_location_loop
```

- Compare **space_cnt** with 0 to check if printing space is necessary or not. If the user selects left as location, the **space_cnt** would be 0 and it will jump to next function instead of printing space.

- **Mov ecx, mov edx,** and **call** are used to print spaces.
- After printing one space it decreases one variable from **space_cnt** using **dec** command.
- Loops back to printing another space through **jmp** command

sq/hex/hg/dia/hrt_col_loop_initialization:

```
sq_col_loop_initialization:
    mov eax, [max_size]
    neg eax
    mov [current_col], eax
```

```
hex_col_loop_initialization:
    mov eax, [max_width]
    neg eax
    mov [current_col], eax            ; put the max width into current_col which is our starting point
```

```
hg_col_loop_initialization:
    mov eax, [max_size]
    neg eax
    mov [current_col], eax              ; sets the horizontal grid
```

```
dia_col_loop_initialization:
    mov eax, [max_size]
    neg eax
    mov [current_col], eax             ; sets the horizontal grid
```

```
hrt_col_loop_initialization:
    mov eax, [max_size]
    neg eax
    mov [current_col], eax             ; X starts at -max_size
```

- **max_size** is inserted into **eax** register to initialize column loop.
- **neg eax** makes the x coordinate negative.
- Data inside **eax** register is moved to **current_col** to calculate columns.

sq/hex/hg/dia/hrt_col_loop:

```
sq_col_loop:
    mov eax, [current_col]
    cmp eax, [max_size]
    jg sq_next_row                      ; jump to next row if its greater
```

```
hex_col_loop:
    mov eax, [current_col]
    cmp eax, [max_width]                ; compares to max width
    jg hex_next_row                     ; jumps to next row if the column is great than max width

    mov ecx, [current_row]
    cmp ecx, 0
    jge hex_get_width                   ; jumps to gather width if the current_row count comes to 0 or greater
    neg ecx
```

```
hg_col_loop:
    mov eax, [current_col]
    cmp eax, [max_size]
    jg hg_next_row
```

```
dia_col_loop:
    mov eax, [current_col]
    cmp eax, [max_size]
    jg dia_next_row
```

```
hrt_col_loop:
    mov eax, [cursrent_col]
    cmp eax, [max_size]
    jg hrt_next_row                     ; if X > max_size, drops down to the next row
```

- The data inside **current_col** is moved back to **eax**.
- **eax** register is compared with **max_size** chosen by the user to calculate how many columns to print.
- Jumps to next row if **max_size** is greater than the value inside the **eax** register.

sq/hex/hg/dia/hrt_print_solid:

```
sq_print_solid:
    mov ecx, draw_character            ; Prints the character user chose
    mov edx, 2
    call print_msg
    jmp sq_next_col
```

```
hex_print_solid:
    mov ecx, draw_character
    mov edx, 2
    call print_msg
    jmp hex_next_col
```

```
; prints the character
hg_print_solid:
    mov ecx, draw_character
    mov edx, 2
    call print_msg
    jmp hg_next_col
```

```
; prints the character
dia_print_solid:
    mov ecx, draw_character
    mov edx, 2
    call print_msg
    jmp dia_next_col
```

```
hrt_print_solid:
    mov ecx, draw_character
    mov edx, 2
    call print_msg
    jmp hrt_next_col
```

- Prints the character using **ecx, edx** and **call** commands.
- **mov edx, 2** uses 2 bytes only, one for the character drawn and other for “Enter” key.
- After that, it jumps to the **next_col** function.

sq/hex/hg/dia/hrt_print_empty:

```
sq_print_empty:
    mov ecx, space                    ; print empty space when needed
    mov edx, 2
    call print_msg
    jmp sq_next_col
```

```
hex_print_solid:
    mov ecx, draw_character
    mov edx, 2
    call print_msg
    jmp hex_next_col
```

```
; prints empty space
hg_print_empty:
    mov ecx, space
    mov edx, 2
    call print_msg
```

```
; prints empty space
dia_print_empty:
    mov ecx, space
    mov edx, 2
    call print_msg
```

```
hrt_print_empty:
    mov ecx, space
    mov edx, 2
    call print_msg
```

- Prints empty space using **ecx, edx** register to gather data and **call** to invoke **sys_write.**

sq/hex/hg/dia/hrt_next_col:

```
sq_next_col:
    inc dword [current_col]             ; moves to the next column
    jmp sq_col_loop
```

```
hex_next_col:
    inc dword [current_col]             ; makes X = X + 1
    jmp hex_col_loop
```

```
; jumps to the next column
hg_next_col:
    inc dword [current_col]
    jmp hg_col_loop
```

```
; jumps to the next column
dia_next_col:
    inc dword [current_col]
    jmp dia_col_loop
```

```
hrt_next_col:
    inc dword [current_col]             ; X = X + 1
    jmp hrt_col_loop
```

- Makes x = x + 1 inside **current_col** variable using **inc** command. Then, jumps to specified **col_loop**.

sq/hex/hg/dia/hrt_next_row:

```
sq_next_row:
    mov ecx, newline
    mov edx, 1
    call print_msg
    inc dword [current_row]             ; moves to the next row
    jmp sq_row_loop
```

```
hex_next_row:
     mov ecx, newline
     mov edx, 1
     call print_msg
     inc dword [current_row]                ; makes Y = Y + 1
     jmp hex_row_loop
```

```
; jumps to the next row
hg_next_row:
    mov ecx, newline
    mov edx, 1
    call print_msg
    inc dword [current_row]
    jmp hg_row_loop
```

```
; jumps to the next row
dia_next_row:
    mov ecx, newline
    mov edx, 1
    call print_msg
    inc dword [current_row]
    jmp dia_row_loop
```

```
hrt_next_row:
    mov ecx, newline
    mov edx, 1
    call print_msg
    inc dword [current_row]             ; Y = Y + 1
    jmp hrt_row_loop
```

- A newline is printed to move onto the next row using **ecx, edx** and **call** commands.
- Makes y = y+1 inside **current_row** variable using **inc.** After this, jumps to **row_loop.**

sq/hex/hg/dia/hrt_shape_done:

```
sq_shape_done:
    ; Reset the terminal to normal color
    mov ecx, c_reset
    mov edx, 4
    call print_msg

    ; Print gap between shapes
    mov ecx, newline
    mov edx, 1
    call print_msg
    ; Decrement total count and jump back to main loop
    dec dword [num_of_shapes]           ; subtracts from the total number of squares
    jmp sq_master_loop
```

```
hex_shape_done:
    mov ecx, c_reset
    mov edx, 4
    call print_msg

    mov ecx, newline
    mov edx, 1
    call print_msg
    call print_msg

    dec dword [num_of_shapes]           ; after finishing printing one shape, subtracts 1 from the total shape
    jmp hex_master_loop
```

```
; finalizes when one shape is done
hg_shape_done:
    mov ecx, c_reset                    ; resets the terminal to default color
    mov edx, 4
    call print_msg

    mov ecx, newline                    ; prints a new line for the next hourglass shape
    mov edx, 1
    call print_msg
    call print_msg

    dec dword [num_of_shapes]           ; after printing one hourglass, it decreases 1 from the total count
    jmp hg_master_loop                  ; then it jumps back to the master loop to print the next hourglass
```

```
; finalizes when one shape is done
dia_shape_done:
    mov ecx, c_reset                    ; resets the terminal color
    mov edx, 4
    call print_msg

    mov ecx, newline                    ; prints gap for the next diamond shape
    mov edx, 1
    call print_msg
    call print_msg

    dec dword [num_of_shapes]           ; subtracts 1 from the shape count
    jmp dia_master_loop                 ; jumps back up for the next shape
```

```
hrt_shape_done:
    mov ecx, c_reset
    mov edx, 4
    call print_msg

    mov ecx, newline
    mov edx, 1
    call print_msg
    call print_msg

    dec dword [num_of_shapes]
    jmp hrt_master_loop
```

- **Mov ecx, c_reset** is used to reset the terminal color after printing a shape.
- **mov edx, 4** is used because ANSI color codes are 4 bytes long.
- **ecx, edx** and **call** is used to print newlines. Here, call is used twice to make 2 lines between shapes.
- After printing a shape and newline it decreases one integer from **num_of_shapes** and jumps to printing the next shape.

2.2.4 CONCENTRIC SQUARE DRAWING ALGORITHM

To draw a concentric square the program compares between |x| and |y| to pick the highest number. After that, it checks for two conditions:

Condition A: If the highest number is even, it will print a character.

Condition B: If the highest number is odd, it will print an empty space.

sq_master_loop:

```
sq_master_loop:
    mov eax, [num_of_shapes]
    cmp eax, 0                          ; dictates how many squares would be drawn
    je main_menu

    call generate_random_color          ; calls the dedicated subroutine for color randomizer

    ; Paints the color
    mov ecx, [chosen_color]
    mov edx, 5                          ; 5 cause the code of the color is usually 5 bytes long
    call print_msg

    ; Row loop initialization
    mov eax, [max_size]
    neg eax                             ; turns the input negative to make the (0,0) coordinates center
    mov [current_row], eax              ; puts the input size into the current row counter
```

- The number of shapes chosen by the user is inserted into **eax** register.
- It then compares itself with 0**(cmp eax, 0)** to decide how many squares must be drawn before it can return to the main menu using **je** command.
- Calls the dedicated randomization function designed to randomize colors.
- Paints the color by putting randomized color into **ecx** register and providing 5 bytes as total length in **edx** register.

sq_get_x:

```
sq_get_x:
    mov ecx, [current_col]              ; grab x
    cmp ecx, 0                          ; checks if x is negative
    jge sq_get_y                        ; if it is 0 or positive, jumps to the next step
    neg ecx                             ; if it is negative, neg flips it to positive
```

- The **ecx** register grabs x coordinate.
- Compares it with 0 to check if x is negative or not.
- If it is 0 or positive, it jumps to get y coordinate.
- If it is negative, it is turned into positive through **neg ecx.**

sq_get_y:

```
sq_get_y:
    mov eax, [current_row]              ; grabs y
    cmp eax, 0                          ; checks if y is negative?
    jge sq_find_biggest_number          ; if it is 0 or positive, jumps to the next step
    neg eax                             ; if it is negative, neg flips it to positive
```

- The **eax** register grabs y coordinate.
- Compares it with 0 to check if y is negative or not.
- If it is 0 or positive, it jumps to finding the biggest number.
- If it is negative, it is turned into positive through **neg eax**.

sq_find_biggest_number:

```
sq_find_biggest_number:
    cmp ecx, eax                        ; compares them
    jle sq_check_even                   ; jumps if ecx is less or equal to eax.
    mov eax, ecx                        ; If ecx is bigger, overwrites eax with ecx.
```

- Compares the coordinates in **ecx** and eax register to find the biggest number through **cmp ecx, eax.**
- If ecx is less than or equal to eax **(jle)**, it jumps to checking if it is even or not.
- **mov eax, ecx** overwrites eax with ecx if ecx is bigger than eax.

sq_check_even:

```
sq_check_even:
    and eax, 1                      ; isolates the Even/Odd bit because every even decimal number in binary ends with 0 and odd numbers end with 1
    jz sq_print_solid               ; jz means jump if zero (even), then prints the symbol
    jmp sq_print_empty              ; otherwise, if it's odd prints a empty space
```

- **and eax, 1** does a binary math trick to calculate even and odd numbers. Every decimal number in binary ends with 0 and odd numbers end with 1. So, it isolates the last binary from the binary digits.
- It prints the symbol if the number is even through **jz sq_print_solid.**
- Otherwise, it prints empty space through **jmp sq_print_empty.**

2.2.5 Hexagon Drawing Algorithm

To draw a hexagon program must calculate the max width using this formula (max_width/w =[(max_size *2) - |y|]. Then, there are two conditions to draw:

Condition A: |x| = w

Condition B: |y| = max_size AND |x| <= w

hex_master_loop:

```
hex_master_loop:
    mov eax, [num_of_shapes]            ; grabs the number on how many hexagon to draw
    cmp eax, 0
    je main_menu                        ; after printing every hexagon required it jumps back to the main menu

    ; applies the choosen color
    mov ecx, [chosen_color]
    mov edx, 5
    call print_msg
    call generate_random_hex_size       ; calls the dedicated subroutine for size randomizer

    ; calculate max width/w = rand_size * 2
    mov eax, [rand_size]
    imul eax, 2
    mov [max_width], eax
```

- Moves the total number of shapes needed to print in **eax** register.
- **cmp eax, 0** adds a limit to draw until it hits 0, then it loops back to the main menu function.
- It applies the chosen color using the same logic previously explained through **chosen_color** variable and **ecx, edx** registers.
- Calculates the max_width formula by putting **rand_size** variable into **eax** register.
- **imul eax, 2** multiplies the value inside **eax** by 2.

- Calculated value is put inside **max_width** variable.

hex_get_width:

```
hex_get_width:
    mov ebx, [rand_size]              ; puts size into ebx
    imul ebx, 2                       ; multiplies by 2
    sub ebx, ecx                      ; subtracts |Y|. the formula is ebx = (rand_size * 2) - |Y|

    mov eax, [current_col]
    cmp eax, 0                        ; checks if x is 0 or positive
    jge hex_check_bounds              ; if greater than 0 or equal, it jumps to the next function
    neg eax                           ; if its negative, "neg" now makes it positive
```

- **rand_size** is put inside **ebx.**
- Multiplies by 2 using **imul ebx, 2.**
- Subtracts |Y| from the calculated integer inside **ecx** through **sub ebx,ecx.**
- Value is moved inside **eax** to be compared with 0. If greater or equal, it jumps to next functions, otherwise the negative value is turned positive using **neg.**

hex_check_bounds:

```
hex_check_bounds:
    cmp eax, ebx                      ; Condition A: Is |X| == W
    je hex_print_solid                ; If yes then prints the character

    cmp ecx, [rand_size]              ; Condition B: Is |Y| == rand_size
    jne hex_print_empty               ; If not on the top or bottom row then print space

    cmp eax, ebx                      ; Condition B: If |Y| == rand_size, now it checks is |X| <= W
    jle hex_print_solid               ; if yes then prints the character
    jmp hex_print_empty               ; otherwise, prints space
```

- Checks if |x| = w by comparing eax and ebx registers**(cmp eax,ebx).** If condition matches, it prints the character.
- Compares **ecx** with **rand_size** to see if |y| = **rand_size.** If not, prints empty space.
- If the previous condition matches, then now it checks whether |x| <= w through **cmp eax, ebx.**
- If matches, prints the character, otherwise prints empty space.

2.2.6 Hourglass Drawing Algorithm

Hourglass checks two conditions to draw:

Condition A: |x| == |y| (draws the diagonal walls: \ and /)

Condition B: |y| == **max_size** AND |x| <= |y| (draws the flat ceiling and floor: ---)

hg_master_loop:

```
hg_master_loop:
    mov eax, [num_of_shapes]           ; checks how many hourglass to draw
    cmp eax, 0                         ; compares if every hourglass asked to draw is finished or not
    je main_menu                       ; loops back to main menu when finished

    ; paints the color chosen to the hourglass
    mov ecx, [chosen_color]
    mov edx, 5
    call print_msg

    call generate_random_hg_location   ; calls the dedicated subroutine for location randomizer
    jmp hg_init_row_loop               ; jumps to row loop initialization since routing is done in subroutine
```

- Uses the same logic explained previously to check how many hourglass needs to be drawn by using **mov, eax**, and **cmp** commands.
- Applies chosen color using the same logic explained previously.
- Calls the location randomization function and jumps immediately to the next loop.

hg_get_x/_y:

```
hg_get_x:
    mov eax, [current_col]             ; gets |X|
    cmp eax, 0
    jge hg_get_y
    neg eax
```

```
hg_get_y:
    mov ecx, [current_row]             ; gets |Y|
    cmp ecx, 0
    jge hg_check_bounds
    neg ecx
```

- Uses same **get_x** and **get_y** absolute value logic as the Concentric Square shape.

hg_check_bounds:

```
hg_check_bounds:
    ;Condition A: |X| == |Y| (draws the diagonal walls: \ and /)
    cmp eax, ecx
    je hg_print_solid

    ; Condition B: |Y| == max_size AND |X| <= |Y| (draws the flat ceiling and floor: ---)
    cmp ecx, [max_size]
    jne hg_print_empty

    cmp eax, ecx
    jle hg_print_solid
    jmp hg_print_empty
```

- Compares **ecx** with **eax** to check if |x| = |y|. If the condition matches, it prints a character.
- Compares **max_size** with **ecx,** if |y| = **max_size**.
- Then, compares **ecx** with **eax** to check if |x| < = |y|. If both conditions are fulfilled, it prints a character. Otherwise, prints an empty space.

2.2.7 Diamond Drawing Algorithm

Diamond drawing algorithm uses only one condition to draw:

Condition: |x| + |y| = **max_size**

dia_master_loop:

```
dia_master_loop:
    mov eax, [num_of_shapes]            ; grabs how many diamonds to draw
    cmp eax, 0
    je main_menu                        ; loops back to main menu when done

    ; paints the chosen color
    mov ecx, [chosen_color]
    mov edx, 5
    call print_msg

    call generate_random_dia_char       ; calls the dedicated subroutine for character randomizer
```

- Checks how many diamonds to draw using **eax, cmp** commands. After printing every shape, it jumps back to the main menu.
- Paints the chosen color using the same logic previously explained.
- Calls randomization function to randomize characters drawn.

dia_get_x/_y:

```
dia_get_x:
    mov eax, [current_col]              ; gets |X|
    cmp eax, 0
    jge dia_get_y
    neg eax
```

```
dia_get_y:
    mov ecx, [current_row]              ; gets |Y|
    cmp ecx, 0
    jge dia_check_bounds
    neg ecx
```

- Uses the same **get_x** and **get_y** absolute value logic as the Concentric Square.

dia_check_bounds:

```
dia_check_bounds:
    ; A diamond's walls perfectly align where |X| + |Y| == max_size
    add eax, ecx                        ; eax = |X| + |Y|
    cmp eax, [max_size]
    je dia_print_solid                  ; prints character if they perfectly equal max size
    jmp dia_print_empty                 ; otherwise leaves it completely hollow
```

- Addition of number inside **eax** and **ecx** is done by using **add** command.
- The **cmp** command checks if the added number is equal to **max_size** variable.
- If equal, it prints a character. If not equal, it prints an empty space.

2.2.8 Heart Drawing Algorithm

Three conditions must be fulfilled to draw a heart. These conditions are:

Condition A: |x| + y <= **max_size**; prints the character.

Condition B: x = 0 OR |x| = **max_size**; prints empty space.

Condition C: |x| <= **max_size**; prints the character.

hrt_master_loop:

```
hrt_master_loop:
    mov eax, [num_of_shapes]
    cmp eax, 0
    je main_menu                        ; Return to menu when done

    ; Paints the color
    mov ecx, [chosen_color]
    mov edx, 5
    call print_msg
```

- Loads the remaining number of shapes using **mov eax, [num_of_shapes]** and compares it with zero using **cmp eax, 0**.
- Jumps to the main menu if no shapes remain, otherwise applies color **using mov ecx, [chosen_color]** and **call print_msg**.

hrt_init_row_loop:

```
hrt_init_row_loop:
    ; STEP 1: SET THE PROPORTIONS
    ; put the max_size into eax, divide it by 2, and make it negative.
    ; Example: if max_size is 6, Y starts at -3.
    mov eax, [max_size]                 ; loads the Size into eax
    xor edx, edx                        ; wipes edx clean
    mov ebx, 2                          ; loads our divisor (2) into a spare register
    div ebx                             ; divides edx:eax by ebx. the answer stays in eax
    neg eax                             ; flips it negative
    mov [current_row], eax              ; saves this as our starting Y coordinate
```

- Initializes the starting Y coordinate by loading **max_size** using **mov eax, [max_size],** dividing it using **div ebx**, and converting it into negative using **neg eax**.
- Stores the result into memory using **mov [current_row], eax**.

hrt_get_x:

```
hrt_get_x:
    mov eax, [current_col]
    cmp eax, 0
    jge hrt_evaluate                    ; if X is 0 or positive, jump to the math
    neg eax                             ; if X is negative, flip it positive. EAX now = |X|
```

- Converts the X value into absolute form by checking its sign using **cmp eax, 0** and applying **neg eax** when needed.

hrt_evaluate:

```
hrt_evaluate:
    mov ecx, [current_row]              ; grabs our raw Y coordinate into ECX
    cmp ecx, 0
    jl hrt_top_half                     ; if Y is negative (< 0), jump to the top half logic
```

- Loads the Y coordinate using **mov ecx, [current_row]** and determines the drawing region using **cmp ecx, 0** and **jl hrt_top_half**.

hrt_bottom_half:

```
hrt_bottom_half:
    ; STEP 2: THE BOTTOM TRIANGLE (Y >= 0)
    ; Math: Checks if |X| + Y <= max_size--
    add eax, ecx                        ; adds |X| and Y together
    cmp eax, [max_size]
    jle hrt_print_solid                 ; If the sum is less than or equal to size, prints the character
    jmp hrt_print_empty                 ; otherwise, it prints empty space
```

- Evaluates the bottom triangle condition by adding coordinates using **add eax, ecx** and comparing with **max_size** using **cmp**.
- Prints a character if valid using **jle hrt_print_solid**, otherwise prints a space using **jmp hrt_print_empty.**

hrt_top_half:

```
hrt_top_half:
    ; STEP 3: THE TOP HUMPS (Y < 0)
    ; First, we check if we are currently scanning the absolute highest row in the grid (Y == -Size/2).
    ; div requires eax, but eax currently holds |X|! We must protect it.
    push eax                            ; saves |X| to the stack
    push edx                            ; saves edx to the stack just in case

    mov eax, [max_size]                 ; loads size into eax for division
    xor edx, edx                        ; wipes edx clean
    mov ebx, 2                          ; loads divisor into ebx
    div ebx                             ; divides edx:eax by ebx. Answer goes to eax.

    mov ebx, eax                        ; Moves our answer (size/2) safely into ebx
    pop edx                             ; restores edx from the stack
    pop eax                             ; restores |X| back into eax!

    neg ebx                             ; makes it negative
    cmp ecx, ebx                        ; is our current Y equal to the top row?
    jne hrt_top_hump_body               ; if no, jump down and just draw the solid body of the hump.
```

- Handles the upper part by calculating the midpoint using **eax, ebx, di**v and comparing it with the current row using **cmp**.
- Redirects to body logic when not on the top row using **jne hrt_top_hump_body**.

hrt_top_cleft_and_corners:

```
hrt_top_cleft_and_corners:
    ; STEP 4: knocks out three specific blocks to round out the shape.
    cmp dword [current_col], 0          ; check if we are at the exact center column (X = 0)
    je hrt_print_empty                  ; if yes, print an empty space (carves the cleft)

    cmp eax, [max_size]                 ; checks if |X| is equal to the max size?
    je hrt_print_empty                  ; if yes, print an empty space! (this carves the rounded shoulders)

    jmp hrt_print_solid                 ; if it's not the cleft and not a shoulder, print a solid block
```

- Creates the heart cleft by checking the center using **cmp [current_col], 0** and removing it using **je hrt_print_empty**.
- Forms rounded edges by checking limits using **cmp eax, [max_size]** and removing them, otherwise fills using **jmp hrt_print_solid**.

hrt_top_hump_body:

```
hrt_top_hump_body:
    ; For all the negative rows below the top row, just draw a solid
    ; block of characters, ensuring not drawing wider than the max_size.
    cmp eax, [max_size]                ; checks if |X| <= max_size or not
    jle hrt_print_solid
    jmp hrt_print_empty
```

- Fills the upper body by checking width using **cmp** and printing using **jle hrt_print_solid**.

3.0 SECTION B: RESEARCH & ANALYSIS

3.1 Task 1: High-Level vs Low-Level Comparison

To effectively analyze the architectural differences between high-level and low-level programming, the NASM assembly shape generator was fully replicated in python language. Both programs show identical output and utilize the same mathematical algorithms to generate distinct shapes. However, the methods they use to interact with system resources and hardware to generate shapes are fundamentally different.

3.1.1 PROGRAMMING LANGUAGE CHARACTERISTICS

The programming language characteristics define how the language controls code syntax, memory management, loop iterations, and overall code readability.

- **Code Volume & Syntax:** In assembly programming, more than 1000+ lines of code were needed for the shape generation program while python needed only 250+ lines of code. Furthermore, every action in assembly is explicit – moving values into registers, calling **sys_write** to write and **int0x80** to trigger actions. Whereas, in python, **print()** abstracts all of that. On top of that, every micro-operation in assembly must be explicitly stated. For example, finding an absolute value in assembly requires checking the negative flag **(cmp ecx, 0)**, utilizing a conditional jump if positive **(jge)**, and turning into positive if the number is negative using **neg.** In python, this multi-step hardware process is compressed into a single built-int function **(abs_col = abs(current_col).**
- **Hardware Proximity:** Assembly directly interacts with OS via system calls (e.g. **sys_read = 3, sys_write = 4**). On the contrary, python input() and print() sit behind the interpreter, making the programmer to never touch the hardware. For example, when the Python code executes **color = random.choice(["\033[31m", "\033[32m", ...]),** the python interpreter silently handles the underlying CPU clock cycles and memory allocation which is done manually in assembly using **rdtsc** and **modulo divison**.
- **Memory Management:** Assembly manually reserves memory with **resb, resd** in **.bss** ram and protects register using **push** & **pop** on every subroutine function call. Python handles all memory allocation automatically through its garbage collector. For example, in python, a string variable named **line = “ “** is created which is then dynamically resized based on

the **x_offset** multiplication and destroyed in the background without any manual intervention. But in assembly, programmers must manually set every value and do manually resize value based on the **x_offset** multiplication to print empty spaces.

Language Characteristics Comparison Table

Characteristics	Assembly	Python
Code Length	1000+ lines – every step explicit	250+ lines – every code concise and readable.
Syntax Style	Register-based, labels, jump instructions	Natural language, indentation-based
Hardware proximity	Direct — int 0x80, rdtsc, .bss section	Abstracted — interpreter handles OS calls
Memory management	Manual — resb, resd, push eax / pop eax	Automatic — garbage collector
Abstraction level	Low — closest to machine code	High — far from the hardware

Figure 1: Programming language characteristics — Assembly vs Python.

3.1.2 DEVELOPMENT EFFICIENCY

Development Efficiency dictates the speed of logically mapping, writing, testing, and debugging of a program.

- **Complexity of Loops:** Looping mechanism in assembly is much more complex than python. To traverse the x-axis in assembly, requires an initialization function, a **cmp** evaluation, a conditional branch **(jg)**, a manual counter increment using **(inc dword[current_col])**, and one unconditional jump **(jmp)**. Python does all this in a single highly readable statement using **for current_col in range(-max_size, max_size + 1):** which simplifies the iteration complexity done by assembly.
- **Input & Error Handling:** Python does all input validation in one function using **get_valid_input()** with **try/except.** However, assembly does the same thing using five separate functions (**error_size, error_location, error_number, error_character, error_color, error_main),** each printing the same duplicate error message and jumping

back to the main loop. This difference shows that input validation inside python code is more simple, effective, and easier to read than assembly language.

- **Debugging & Testing:** When certain error occurs inside python program, the terminal shows descriptive runtime error like **ValueError, IndexError** etc. Assembly produces a segmentation fault message in the terminal whenever something goes wrong inside the assembly code. This forces the programmer to trace register value manually using a debugger like gdb. This leads to the conclusion that assembly program is very difficult to debug whereas python debugging is effortless compared to this.

Development Efficiency Comparison Table

Factor	Assembly	Python
Writing speed	Slow — manual register management throughout	Fast — high-level constructs and libraries
Loop structure	5+ jump labels per loop (init, loop, next_col, next_row, done)	Two lines: for row in range(...): for col in range(...)
Input handling	sys_read + manual byte count check (cmp eax, 2)	input() built-in — one line
Error handling	6 separate error labels, each jumps back to its prompt	One get_valid_input() with try/except
Debugging	No runtime messages — requires gdb register tracing	Descriptive exceptions (ValueError, TypeError)
Code reuse	Shared subroutines via call/ret (print_msg, read_input)	Functions, default parameters, return values
Testing cycle	Assemble → link → run (nasm + ld every change)	Run instantly: python3 shapes.py

Figure 2: Development Efficiency — Assembly vs Python.

3.1.3 OVERALL SUITABILITY

Python has less code than Assembly while generating identical output in the terminal. Additionally, python is easier to write, debug, and maintain overall than Assembly coding. For a shape generator where performance is not an issue, Python is clearly the suitable choice for this program.

While Assembly programming demonstrates hardware control and low-level programming principles, Python is more suitable due to simplified code syntax, built-in error handling, and significantly shorter program development period. Both programs produce identical outputs ensuring that the algorithm used in both programs are similar. Thus, the difference lies in the effort of producing the program while implementing and maintaining the code where Python wins without any doubt.

3.2 Task 2: Branch Prediction & Caching

This task analyzes how unpredictable inputs and randomized parameters affect the overall program performance and efficiency. It then proposes how artificial intelligence or machine learning could enhance system's adaptability and efficiency.

3.2.1 IMPACT ON RANDOM EVENTS ON SYSTEM PERFORMANCE

The shape generation program uses five distinct randomization mechanisms to randomize outputs – one per shape. These randomization factors are implemented via **rdtsc** in Assembly and **random** in Python to produce unpredictable output at the terminal. This section examines how these unpredictable behaviors affect branch prediction, cache usage, and overall program responsiveness.

- **Random Input Parameters:** In Assembly, **rdtsc** reads CPU clock cycles. These clock cycle changes every nanosecond, making the remainders "7" for color or "3" for location genuinely unpredictable. The CPU's branch predictor cannot identify which branch **(e.g. rand_set_red, rand_set_left)** will be taken. As a result, each shape drawn causes a misprediction of roughly 15-20 clock cycles (Patterson & Hennessy, 2016). In Python, **random.choice()** and **random.randint()** use a Mersenne Twister algorithm internally (Matsumoto & Nishimura, 1998). While statistically better than **rdtsc** modulo in Assembly, it still produces unpredictable results which prevents Python interpreter from optimizing the conditional branches inside the program.

- **Random Place Shapement:** The Hourglass randomizes **x_offset** between 0, 20 and 40 using **generate_random_hg_location.** It means the location loop runs different number of iterations for every shape drawn. Thus, the CPU cannot predict the exact iteration count. However, Python abstracts the randomness in one line using **x_offset = random.choice([0, 20, 40]),** but still produces unpredicted results in runtime.
  The same way, number of hearts (1-9) drawn at the terminal is also randomized using the same logic in both languages. This means that the total execution path is unknown to the user. A user who runs the same program twice will get different number of drawings affecting total rendering time.
- **Unpredictable User Behaviour:** The **read_input** subroutine always reads up to 16 bytes **(mov edx, 16)**, but the OS delivers the length until the user pressed "Enter" key. The program rejects any input over 2 bytes through **cmp eax, 2** which validates invalid input. But since users behave randomly, the error branch fills itself randomly.
  In Python, **get_valid_input()** handles this with a **try/except ValueError** loop. This invalidates any string with more than 1 characters and any character that is not an integer. The exception path is rare in normal use which leads to Python giving slightly better performance than the Assembly equivalent.
- **Cache Behaviour:** The Assembly program uses small set of **.bss** variables (e.g. **max_size, x_offset, current_col, current_row)** across all five shapes. These variables are reused multiple times. Because the same memory locations are repeatedly read and written, they stay in L1 data cache which is beneficial for the program performance. Meanwhile, in python, the drawing functions use local variables like **current_col, line, abs_col** etc. that are all allocated in different functions. Python's interpreter and OS decide where these variables live, so the programmer has no control over cache behaviour.

3.2.2 AI/ML ENHANCEMENT PROPOSALS

The current system uses static algorithms and fixed randomization using CPU clock cycles. Artificial Intelligence or Machine Learning can extend its capabilities from auto generating new shapes to allocating program resources before execution. Four proposals are outlined below:

- **Automatic Shape Design Generation:** A Variational Autoencoder (VAE) or Generative Adversarial Network (GAN) trained on ASCII grid representations of shapes could learn

the boundary rules algorithmically (Goodfellow et al., 2016). Instead of a programmer writing |**X**| + |**Y**| == **max_size** for a diamond, the model would learn this condition from labelled examples. Consequentially, new shapes can be generated without any manual coding as the model sets the conditions while the drawing loop executes it.

- **Drawing Loop Optimization:** A Reinforcement Learning (RL) agent can minimize total iterations in a program to provide optimal and quick results. The agent would think computation speed and lower memory usage as its reward. For example, fewer total_print_msg functions would be the reward for this agent. Additionally, it might be able to skip entire rows early for the Diamond shape where most cells are empty. This will lead to adding a smarter exit condition than the current brute force grid scan from **-max_size** to +**max_size.**
- **Resource Demand Prediction:** A regression model trained in inputs (size, number of shapes, location, color, symbol) can be implemented to predict execution time and memory usage before runtime occurs. Since number of shapes is randomized in heart shape, user cannot predict how many shapes it will draw. Furthermore, users cannot project how much time it will take to print randomized outputs according to the program. To solve this, a trained model can display an estimated time "2.3 seconds" before committing to the draw loop.
- **Adaptive Randomization:** A Markov model can identify the preferred randomization combination a user repeatedly selects (e.g. always picking size 3 with green color). The model will recognize this and will set the parameter to randomize size from 7 to 9 while generating different colors than green to ensure variety. This way more randomized output can be generated than the given program. This turns the crude **rdtsc** randomization into intelligent, session-aware system.
- **Predictive Resource Allocation via Neural Networks:** A recurrent neural network (RNN) can be implemented to observe user input patterns over time. Normally, a computer tries to blindly load the code in advance, guessing what the user will choose. Instead of this, RNN can learn user habits and typos. For example, if the user prints hexagon many times or prints using I symbol, the program will load the hexagon and I symbol code in advance before the user presses "Enter" key. This will make the program reduce delays and feel instantly responsive.

4.0 CONCLUSION

This report demonstrates the design and integration of the menu drive shape generator program using 32-bit NASM Assembly Programing. The implementation analysis showed how low-level languages directly interact with hardware through system calls, manual memory allocation, and CPU-level randomization utilizing CPU clock cycles. In section b, it is proven that both Assembly and Python can generate identical programs with similar output. However, Python has proven to be more suitable than Assembly for this program due to its simple code syntax, easier debugging, and built-in error handling. The random analysis revealed how unpredictable inputs and rdtsc parameters influence branch mispredictions and variable cache behaviour which affects program efficiency. The proposed AI and ML enhancements including generative shape design, reinforcement learning loop optimization, regression-based resource prediction, adaptive randomization, and recurrent neural network demonstrates the possibilities of how the current static program can be improved into an intelligent, self-adaptive program.

5.0 REFERENCES

- Python Software Foundation. (2024). *random — Generate pseudo-random numbers*. https://docs.python.org/3/library/random.htm
- Patterson, D. A., & Hennessy, J. L. (2016). *Computer organization and design ARM edition: The hardware software interface*. Morgan Kaufmann.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. The MIT Press.